设备树详解

2016年11月24日 16:53:11 XiaoBaWu 阅读数：11558 标签： linux 更多

版权声明：转载需注明出处，若有不足，欢迎指正。 https://blog.csdn.net/qq_28992301/article/details/53321610

设备树详解

在Linux3.x版本后，arch/arm/plat-xxx和arch/arm/mach-xxx中，描述板级细节的代码（比如platform_device、i2c_board_info等）被大量取消，取而代之的是设备树，其目录位于arch/arm/boot/dts

1.设备树的组成

1个dts文件+n个dtsi文件，它们编译而成的dtb文件就是真正的设备树

- soc厂商会把soc公共的特性和多块开发板公用的特性提炼为dtsi，而dts则负责描述某个具体的产品（开发板）的特性。dts直接或间接的包含多个dtsi（类似于c语言的头文件），就体现了一个完整的产品（开发板）所有的特性。以solidrun公司的hummingboard为例，其组成为

```
imx6dl-hummingboard.dts
        |_imx6dl.dtsi
        |   |_imx6qdl.dtsi
        |_imx6qdl-microsom.dtsi
        |_imx6qdl-microsom-ar8035.dtsi
```

- 此外，dts/dtsi兼容c语言的一些语法，能使用宏定义，也能包含.h文件

2.设备树的结构

下面分别是是imx6dl-hummingboard.dts以及imx6dl.dtsi文件，我们以它们为例来分析，不难发现dts文件内容很少，只有一些板级的特征，大部分公共的硬件描述都在dtsi文件中

- imx6dl-hummingboard.dts 文件节选

```
/dts-v1/;
#include "imx6dl.dtsi"
#include "imx6qdl-microsom.dtsi"
#include "imx6qdl-microsom-ar8035.dtsi"

/ {
    model = "SolidRun HummingBoard DL/Solo";
    compatible = "solidrun,hummingboard", "fsl,imx6dl";

    ir_recv: ir-receiver {
        compatible = "gpio-ir-receiver";
        gpios = <&gpio1 2 1>;
        pinctrl-names = "default";
        pinctrl-0 = <&pinctrl_hummingboard_gpio1_2>;
    };

    regulators {
        compatible = "simple-bus";

        reg_3p3v: 3p3v {
```

```
            compatible = "regulator-fixed";
            regulator-name = "3P3V";
            regulator-min-microvolt = <3300000>;
            regulator-max-microvolt = <3300000>;
            regulator-always-on;
        };
    }

&i2c1 {
    pinctrl-names = "default";
    pinctrl-0 = <&pinctrl_hummingboard_i2c1>;

    rtc: pcf8523@68 {
        compatible = "nxp,pcf8523";
        reg = <0x68>;
    };
};
```

- imx6dl.dtsi文件节选

```
/ {
    aliases {

    /*省略无关代码*/
    }
    soc {
        #address-cells = <1>;
        #size-cells = <1>;
        compatible = "simple-bus";
        interrupt-parent = <&intc>;
        ranges;

        /*省略无关代码*/

        timer@00a00600 {
            compatible = "arm,cortex-a9-twd-timer";
            reg = <0x00a00600 0x20>;
            interrupts = <1 13 0xf01>;
            clocks = <&clks IMX6QDL_CLK_TWD>;
        };

        aips-bus@02000000 { /* AIPS1 */
            compatible = "fsl,aips-bus", "simple-bus";
            #address-cells = <1>;
            #size-cells = <1>;
            reg = <0x02000000 0x100000>;
            ranges;

            /*省略无关代码*/

            gpio1: gpio@0209c000 {
                compatible = "fsl,imx6q-gpio", "fsl,imx35-gpio";
                reg = <0x0209c000 0x4000>;
                interrupts = <0 66 IRQ_TYPE_LEVEL_HIGH>,
                         <0 67 IRQ_TYPE_LEVEL_HIGH>;
                gpio-controller;
                #gpio-cells = <2>;
                interrupt-controller;
```

```
                #interrupt-cells = <2>;
            };

            /*省略无关代码*/

            i2c1: i2c@021a0000 {
                #address-cells = <1>;
                #size-cells = <0>;
                compatible = "fsl,imx6q-i2c", "fsl,imx21-i2c";
                reg = <0x021a0000 0x4000>;
                interrupts = <0 36 IRQ_TYPE_LEVEL_HIGH>;
                clocks = <&clks IMX6QDL_CLK_I2C1>;
                status = "disabled";
            };

        };
            /*省略无关代码*/
    };
};
```

基本构造

- {}包围起来的结构称之为节点，dts中最开头的 `/ {}` ，称为根节点。节点的标准结构是xxx@yyy{...}，xxx是节点的名字，yyy则不是必须的，其值为节点的地址（寄存器地址或其他地址），比如 `i2c1: i2c@021a0000` 中的就是一个i2c控制器的寄存器基地址， `rtc: pcf8523@68` 中的就是这个rtc设备的i2c地址

属性：地址

- 有关节点的地址，比如 `i2c@021a0000` ，虽然它在名字后面跟了地址，但是正式的设置是在reg属性中设置的比如： `reg = <0x021a0000 0x4000>;` reg的格式通常为 `<address length>` ，0x021a0000是寄存器基地址，0x4000是长度。address 和length的个数是可变的，由父节点的属性 `#address-cells` 和 `#size-cells` 决定，比如节点 `i2c@021a0000` 的父节点是 `aips-bus@02000000` ，其 `#address-cells` 和 `#size-cells` 均为1，所以下面的i2c节点的reg属性就有一个address 和length，而i2c节点本身 `#address-cells` 和 `#size-cells` 分别为1和0，所以其下的 `rtc: pcf8523@68` 的reg属性就只有一个0x68（i2c地址）了

属性：兼容性

- 如果一个节点是设备节点，那么它一定要有compatible（兼容性），因为这将作为驱动和设备（设备节点）的匹配依据，compatible（兼容性）的值可以有不止一个字符串以满足不同的需求，详见下一节。而根节点的compatible也是非常重要的，也就是 `"fsl,imx6dl"` 这个字符串，因为系统启动后，将根据根节点的compatible来判断cpu信息，并由此进行初始化

属性设置的套路

- 一般来说，每一种设备的节点属性设置都会有一些套路，比如可以设置哪些属性？属性值怎么设置？那怎么知道这些套路呢，有两种思路
  - 第一种是抄类似的dts，比如我们自己项目的平台是4412，那么就可以抄exynos4412-tiny4412.dts、exynos4412-smdk4412.dts这类相近的dts
  - 第二种是查询内核中的文档，比如 `Documentation/devicetree/bindings/i2c/i2c-imx.txt` 就描述了imx平台的i2c属性设置方法； `Documentation/devicetree/bindings/fb` 就描述了lcd、lvds这类属性设置方法

节点之间的联系

- 节点与节点之间的关联，通常通过“标号引用”和“包含”来实现
  - 所谓标号引用，就是在节点名称前加上标号，这样设备树的其他位置就能够通过&符号来调用/访问该节点，比如上面代码ir_recv节点中的gpio属性，就引用了gpio1标号处的节点
  - 包含则是最基本的方式，比如我们要在i2c1接口添加一个i2c外设，那么就必须要在i2c1下面添加一个节点，比如上面代码中的 `rtc: pcf8523@68 {}`

- 标号引用常常还作为节点的重写方式，比如下面代码是imx6qdl.dtsi中定义的i2c节点，而前面imx6dl-hummingboard.dts中的&i2c1，就是对i2c1标号处节点的一次重写，在其内部添加了一个rtc设备
- 如果一个节点是属性节点（即仅仅是作为属性被其他节点调用），那么它定义在哪里其实无所谓，重要的是调用的位置，比如lcd屏幕的时序，其实我们完全可以把它定义在其他犄角旮旯，然后在lcd节点下用&来调用它，这也是可以的。这有点类似于函数：在哪定义不重要，重要的是在哪调用

3.内核（驱动）与节点的匹配

首先，内核必须要知道dtb文件的地址，这由U-boot来告诉内核，详见U-boot引导内核流程分析 第6节。只要内核知晓了dtb文件的地址，那么驱动就可以通过一些API任意获取设备树的内部信息

- 对于3.x版本之后的内核，platform、i2c、spi等设备不再需要在mach-xxx中注册，驱动程序将直接和设备树里的设备节点进行配对，是通过设备节点中的compatible（兼容性）来与设备节点进行配对的，这里只做简单介绍，具体的应用详见 基于i2c子系统的驱动分析、 基于platform总线的驱动分析
- 这里以pcf8523驱动为例，只要驱动中的of_match_table 中的compatible 值和设备节点中的compatible 相匹配，那么probe函数就会被触发。不仅i2c是这样，platform、spi等都是这个原理

```
/*定义的of_match_table*/
static const struct of_device_id pcf8523_of_match[] = {
    { .compatible = "nxp,pcf8523" },
    { }
};

/*driver 结构体中的of_match_table*/
static struct i2c_driver pcf8523_driver = {
    .driver = {
        .name = DRIVER_NAME,
        .owner = THIS_MODULE,
        .of_match_table = of_match_ptr(pcf8523_of_match),
    },
    .probe = pcf8523_probe,
    .id_table = pcf8523_id,
};
```

- i2c和spi驱动还支持一种“别名匹配”的机制，就以pcf8523为例，假设某程序员在设备树中的pcf8523设备节点中写了 `compatible = "pcf8523";` ，显然相对于驱动id_table中的 `"nxp,pcf8523"` ，他遗漏了nxp字段，但是驱动却仍然可以匹配上，因为别名匹配对compatible中字符串里第二个字段敏感

4.常见属性的设置与获取

当修改或编写驱动时，常常需要修改gpio、时钟、中断等等参数，以前都是在mach-xxx中的device设置的，现在则要在节点里设置，然后驱动用特殊的API来获取

- 属性的获取常常在probe函数中进行，但是获取属性之前，最重要的是，确定哪个节点触发了驱动。如果一个驱动对应多个节点，那驱动可以通过 `int of_device_is_compatible(const struct device_node *device, const char *name)` 来判断当前节点是否包含指定的compatible（兼容性）

gpio的设置与获取

```
/*imx6dl.dtsi中gpio1控制器的定义节点*/
gpio1: gpio@0209c000 {
    compatible = "fsl,imx6q-gpio", "fsl,imx35-gpio";
    reg = <0x0209c000 0x4000>;
    interrupts = <0 66 IRQ_TYPE_LEVEL_HIGH>,
             <0 67 IRQ_TYPE_LEVEL_HIGH>;
    gpio-controller;

```

```
    #gpio-cells = <2>;
    interrupt-controller;
    #interrupt-cells = <2>;
};

/*imx6qdl-sabreauto.dtsi中某个设备节点*/
max7310_reset: max7310-reset {
    compatible = "gpio-reset";
    reset-gpios = <&gpio1 15 1>;
    reset-delay-us = <1>;
    #reset-cells = <0>;
};
```

- 一般来说，我们把gpio属性的名字起为 `xxx-gpios` （xxx我们可以随便起），这样驱动才能通过特定API从识别该属性，并转换成具体的gpio号
- 该设备节点中设置了 `reset-gpios = <&gpio1 15 1>;` 这格式是什么意思呢？ `&gpio1 15` 引用了gpio1节点，故此处含义为gpio1_15这个引脚；最后一个参数1则代表低电平有效，0则为高电平有效。至于gpio1_15具体对应哪个引脚，在imx6的手册上都有详细描述
- 其实最后一个参数（高低电平有效）不是必须的，因为gpio1节点中设置了 `#gpio-cells = <2>;` ，所以才有两个参数；某些soc的gpio节点中会设置为 `#gpio-cells = <1>;` ，那么可以不写最后一个参数
- 驱动一般通过以下接口获取上面节点中gpio的属性。该函数第一个参数是节点，一般可以在传入probe的参数中间接获得；第二个参数是gpio属性的名字，一定要和节点属性中的 `xxx-gpios` 相同；最后一个是编号index，当节点中有n个同名的xxx-gpios时，可以通过它来获取特定的那个gpio，同一节点中gpio同名情况很少存在，所以我们都把index设为0

```
gpio = of_get_named_gpio(node, "reset-gpios", index);
```

- 在dts和驱动都不关心gpio名字的情况下，也可直接通过以下接口来获取gpio号，这个时候编号index就十分重要了，可以指定拿取节点的第index个gpio属性

```
gpio = of_get_gpio(node, index);
```

中断的设置与获取

假设某设备节点需要一个gpio中断

```
/*先确定中断所在的组*/
interrupt-parent = <&gpio6>;

/*表示中断，GPIO6中的第 8 个IO，2为触发类型，下降沿触发*/
interrupts = <8 2>;

```

- 而在驱动中使用 `中断号 =irq_of_parse_and_map(node, index)` 函数返回值来得到中断号

自定义属性的设置与获取

所谓的自定义属性，有点类似于老内核中的platform_data，我们在设备节点中可以随意添加自定义属性，比如下面这个节点里面的属性都是我们自己定义的

```
reg_3p3v: 3p3v {
    compatible = "regulator-fixed";
    regulator-name = "3P3V";
    regulator-min-microvolt = <3300000>;

```

```
    regulator-max-microvolt = <3300000>;
    regulator-always-on;
};
```

- 针对32位整形的属性，比如上面的regulator-min-microvolt，可以利用下面这个API来获取属性值，第一个参数是节点，第二个参数是属性名字，第三个是输出型参数（把读出来的值放进去）

```
of_property_read_u32(node, "regulator-min-microvolt", &microvolt);
```

- 类似的读取数值的API还有：

```
int of_property_read_u8(const struct device_node *np, const char *propname, u8 *out_value)

int of_property_read_u16(const struct device_node *np, const char *propname, u16 *out_value)

```

- 下列API可检查节点中某个属性是否存在，存在则返回true，不存在则返回false

```
bool of_property_read_bool(const struct device_node *np, const char *propname)
```

- 当节点中存在字符串时，可以像下面那样读取，比如我们读取前面reg_3p3v节点中的字符串

```
of_property_read_string(node, "regulator-name", &string)
```

- 当节点中存在数组时，可以像下面那样读取

```
/*带有数组的某个节点*/
L2: cache-controller@1e00a000 {
    compatible = "arm,pl310-cache";
    arm,data-latency = <1 1 1>;
    arm,tag-latency = <1 1 1>;
};

/*驱动中使用API来读取数组， &data为输出型参数*/
of_property_read_u32_array(node, "arm,pl310-cache", &data, ARRAY_SIZE(data));
```